

INTERVIEW PREPARATION GUIDE

Complete Technical Deep-Dive for

AI RAG Chatbot & FitAI Pro

Candidate	Domain	Level
Kokkerala Gnaneswara Anjani Prasad	Generative AI / Full Stack	Fresher / Entry Level

GitHub: GNANESWARKOKKIRALA | Portfolio:

gnaneswarkokkerala-portfolio.netlify.app

B.Tech CSE 2026 | Tirumala Engineering College, Narasaraopet, AP

Co-founder: GS Tech Solutions

Table of Contents

- 1. **AI RAG Chatbot — Complete Project Breakdown**
 - 1.1 Project Overview & Elevator Pitch
 - 1.2 Tech Stack with Justification
 - 1.3 System Architecture & Data Flow
 - 1.4 Core Modules — Deep Dive
 - 1.5 Key Algorithms & Design Decisions
 - 1.6 Interview Q&A; — RAG Chatbot
- 2. **FitAI Pro — Complete Project Breakdown**
 - 2.1 Project Overview & Elevator Pitch
 - 2.2 Tech Stack with Justification
 - 2.3 System Architecture & Data Flow
 - 2.4 Database Design — 11-Table Schema
 - 2.5 AI Engine — Feature Breakdown
 - 2.6 Interview Q&A; — FitAI Pro
- 3. **Cross-Project Comparison & Common Interview Topics**
- 4. **Behavioral & HR Questions**
- 5. **Quick Revision Cheat Sheet**

Chapter 1: AI RAG Chatbot

Groq + LLaMA 3.3 70B + ChromaDB + SQLite + Streamlit

1.1 Project Overview & Elevator Pitch

AI RAG Chatbot is a document-based question answering system. Users upload their own files — PDF, DOCX, TXT, CSV, or Markdown — and ask questions about them. The system answers strictly from the uploaded documents, not from the model's training data, which prevents hallucination. Key highlights: multi-user authentication with per-user chat history, real-time streaming responses, source attribution on every answer showing which document and chunk was used, and support for uploading multiple documents simultaneously. The UI is built with Streamlit and deployed on Streamlit Cloud.

The entire RAG pipeline is custom-built without any LangChain abstraction — 6 independent modules covering document loading, chunking, embedding, vector storage, retrieval, and LLM generation. LLM inference runs on Groq API (LLaMA 3.3 70B) delivering ~500 tokens/sec for near-instant streaming. Document embeddings are generated locally using sentence-transformers (all-MiniLM-L6-v2) producing 384-dimensional vectors at zero API cost, stored in ChromaDB with a cosine HNSW index for fast approximate nearest neighbour search. User accounts and chat history are persisted in SQLite with bcrypt-hashed passwords. Temperature is fixed at 0.2 for grounded, deterministic answers.

One-Line Pitch (Say This First in Interview)

■ *"I built a custom RAG chatbot from scratch — no LangChain — using Groq's LLaMA 3.3 70B, ChromaDB for vector search, and sentence-transformers for local embeddings. I implemented every stage of the pipeline myself: document loading, word-based chunking with overlap, 384-dimensional embedding, cosine similarity retrieval, context injection, and streaming generation — so I can explain exactly why each design choice was made."*

Problem Statement

Large Language Models like ChatGPT are trained on public data up to a fixed cutoff date — they cannot answer questions about private, internal, or domain-specific documents. RAG solves this by retrieving the most relevant passages from the user's uploaded files at query time and injecting them as context into the prompt. The model is explicitly instructed to respond only from that context and to acknowledge when the answer is not found — eliminating hallucination on document-specific queries.

Key Features — Full List

- Multi-document upload and management: PDF, DOCX, TXT, CSV, MD — up to multiple files simultaneously

- Custom 6-stage RAG pipeline built from scratch: Load → Chunk → Embed → Store → Retrieve → Generate
- Per-user authentication: bcrypt password hashing, SQLite user table, Streamlit session state
- Persistent chat history: per-user, per-session messages saved to SQLite, restored on login
- Real-time streaming responses: LLaMA 3.3 70B streams token-by-token via Groq API with live cursor
- Source attribution: every AI answer appends the document filenames and chunk indices used
- Token-limit management: trim_history() keeps last 20 messages to prevent 128k context overflow
- Document management panel: sidebar shows all uploaded docs with chunk count and delete button
- Metrics dashboard: live chunk count and document count displayed as metric cards in sidebar
- Dark glassmorphism UI: custom CSS in Streamlit with gradient headers, Inter font, dark cards
- Browser voice input: Web Speech API integration via streamlit-mic-recorder for hands-free queries
- Duplicate detection: checks existing filenames before re-processing the same document
- Error handling: catches rate limit (429), token overflow, and general API errors with user messages
- Reset functionality: full store reset clears ChromaDB, SQLite history, and document records atomically

Project Statistics at a Glance

Lines of Code	Python Modules	Pipeline Stages	Supported Formats	LLM Parameters
~900+	10	6	5 (PDF/DOCX/TXT/CSV/MD)	70 Billion

1.2 Tech Stack — Every Choice Justified

Technology	Role	Why This Choice
Groq API	LLM Inference	Ultra-fast inference at ~500 tok/s vs OpenAI's ~50 tok/s. LLaMA 3.3 70B is open-source backbone with 128k context window. Free tier sufficient for development.
LLaMA 3.3 70B	Language Model	Meta's best open-source model. 70B parameters give near-GPT-4 quality. Versatile variant handles both RAG and general Q&A.; temperature=0.2 for deterministic factual answers.
ChromaDB 0.5.5	Vector Database	Embedded vector DB, no server needed. Supports cosine similarity (HNSW index). PersistentClient saves vectors to disk across restarts. Perfect for local/demo deployments.
sentence-transformers	Embedding Model	all-MiniLM-L6-v2 produces 384-dim embeddings. 80MB model runs CPU-only. Singleton pattern prevents reloading on every request. Avoids paid embedding API costs.
Streamlit 1.36	Frontend / UI	Python-native web UI — no React/JS needed. st.chat_message for proper chat rendering. st.file_uploader handles multi-format uploads. Rapid iteration during development.
SQLite	Relational Storage	Zero-config persistent storage. Stores users, documents, and chat history. bcrypt hashes stored as TEXT. No PostgreSQL/MySQL overhead for demo scale.
bcrypt 4.2.0	Auth / Security	Industry-standard password hashing with adaptive cost factor. Never stores plaintext. checkpw() for constant-time comparison preventing timing attacks.

pdfplumber	PDF Parsing	Better table + layout extraction vs PyPDF2. Maintains reading order. Handles multi-column PDFs. python-docx for DOCX, pandas for CSV.
python-dotenv	Config	Keeps API keys out of source code. .env.example ships as documentation. Standard 12-factor app pattern.

1.3 System Architecture & Data Flow

The RAG pipeline follows a strict unidirectional data flow across 6 stages:

Stage	Module	What Happens
Stage 1 — Load	rag/loader.py	File type detection → text extraction (pdfplumber/python-docx/pandas)
Stage 2 — Chunk	rag/chunker.py	Split text into 500-word windows with 50-word overlap
Stage 3 — Embed	rag/embedder.py	SentenceTransformer (all-MiniLM-L6-v2) → 384-dim float vectors
Stage 4 — Store	rag/vector_store.py	ChromaDB PersistentClient with cosine HNSW index
Stage 5 — Retrieve	rag/retriever.py	Embed query → cosine similarity search → top-K chunks
Stage 6 — Generate	llm/groq_client.py	System prompt + context + trimmed history → LLaMA stream

Authentication Architecture

Action	Code Location	Mechanism
User registers	auth/auth.py → create_user()	bcrypt.hashpw() → INSERT INTO users
User logs in	auth/auth.py → authenticate_user()	bcrypt.checkpw() → session state

Session persists	utils/session.py	st.session_state with user_id, username
History isolated	utils/helpers.py → save_message()	username-scoped SQLite queries

1.4 Core Modules — Deep Dive

Chunking Strategy — chunker.py

The chunker uses a sliding-window word-based approach: 500-word chunks with 50-word overlap. Why word-based vs character-based? Preserves semantic units (sentences don't get split mid-word). Why overlap? Ensures context at chunk boundaries isn't lost — a concept spanning two windows appears in both, improving retrieval recall.

```
chunk_text(text, chunk_size=500, overlap=50) → List[str]
```

The sliding formula: $\text{start} += (\text{chunk_size} - \text{overlap})$ → next window starts 450 words in. Empty chunks are filtered. For a 10,000-word document this produces ~21 chunks.

Embedding Strategy — embedder.py

Singleton pattern for model loading: `_model` is a module-level `None` that gets initialized once. This is critical in Streamlit where the script re-runs on every user interaction — without the singleton, the 80MB model would reload on every query, adding 3-5 seconds of latency.

Model: all-MiniLM-L6-v2 → 384-dimensional dense vectors. Trained on 1B sentence pairs. Balances speed (CPU inference ~20ms per batch) with quality (MTEB benchmark top-10).

Vector Store — ChromaDB Operations

`add_chunks()`: Generates IDs as `doc_id_chunk_0`, `doc_id_chunk_1`... Stores metadata (source filename, `doc_id`, `chunk_index`) alongside each embedding. `query_chunks()`: Embeds the user query → cosine similarity search → returns top-K hits with relevance score = $1 - \text{cosine_distance}$.

Context Window Management — groq_client.py

LLaMA 3.3 70B has a 128k token context window but the Groq free tier enforces lower limits. The `trim_history()` function keeps the last 20 messages (10 user + 10 assistant exchanges). An orphan-check removes leading assistant messages to maintain valid user/assistant alternation. Temperature is set to 0.2 for RAG (factual, grounded) vs higher for general chat.

1.5 Key Design Decisions

Decision: No LangChain

Full control over every pipeline step. LangChain abstracts chunking, embedding, retrieval — hiding bugs and performance bottlenecks. Building from scratch demonstrates deeper understanding of RAG internals for interviews.

Decision: ChromaDB over Pinecone/Weaviate

No API key, no cloud dependency, fully embedded. PersistentClient survives server restarts. HNSW index gives $O(\log n)$ approximate nearest neighbor search. Perfect for demo/portfolio without cloud costs.

Decision: SQLite over PostgreSQL

Zero-config, file-based. Single fitai.db or chat_history.db file. row_factory=sqlite3.Row gives dict-like access. Foreign keys enforced with PRAGMA.

Decision: Streamlit over Flask/FastAPI

10x faster to prototype chat UI. Native chat_message widget. File uploader built-in. Sidebar layout maps perfectly to the document management use case.

Decision: sentence-transformers over OpenAI embeddings

No API cost per embedding call. Runs fully offline. 384-dim vectors are smaller and faster to store/search than 1536-dim OpenAI vectors for this scale.

1.6 Interview Q&A; — AI RAG Chatbot

Q: What is RAG and why did you use it?

RAG stands for Retrieval-Augmented Generation. Instead of relying on the LLM's static training data, RAG fetches relevant document chunks at query time and passes them as context to the LLM. This solves the 'hallucination' problem — the model answers from your actual documents, not invented facts. I used it because users need to query their own private documents, not general internet knowledge.

Q: Explain your chunking strategy and why you chose those parameters.

I use a sliding window of 500 words with 50-word overlap. 500 words fits well within the embedding model's optimal input range (256-512 tokens) without losing semantic coherence. The 50-word overlap ensures concepts at chunk boundaries aren't lost — if a key fact spans two windows, both chunks will contain part of it, improving retrieval recall. Word-based splitting preserves sentence structure better than character-based splitting.

Q: How does cosine similarity work in your vector search?

Cosine similarity measures the angle between two vectors in high-dimensional space, ignoring their magnitude. If my query embeds to vector Q and a chunk embeds to vector C , $\text{cosine_similarity} = (Q \cdot C) / (|Q||C|)$. A score of 1.0 means identical direction (same meaning), 0.0 means orthogonal (unrelated). ChromaDB uses HNSW (Hierarchical Navigable Small World) graph for approximate nearest neighbor search in $O(\log n)$ time. I store $1 - \text{distance}$ as the relevance score.

Q: How do you prevent the LLM from making up answers?

Three mechanisms: (1) System prompt explicitly says 'Answer strictly using the context below. If the answer is not in the context, say I couldn't find relevant information.' (2) Retrieved chunks are formatted as [Source 1: filename.pdf] text... so the model sees provenance. (3) Sources are appended to every response, so the user can verify. Temperature is set to 0.2 for determinism.

Q: How did you handle token limits?

LLaMA 3.3 70B has a 128k token context, but Groq's free tier has lower RPM limits. My `trim_history()` keeps only the last 20 messages (10 exchanges) to stay well within limits. It also checks that the trimmed history doesn't start with an orphan assistant message, which would cause an API validation error. Error handling catches `rate_limit` and `context_length` errors with user-friendly messages.

Q: Why did you use bcrypt over MD5/SHA256?

MD5 and SHA256 are fast hashing algorithms — good for file integrity, bad for passwords. An attacker can compute billions of SHA256 hashes per second with a GPU. bcrypt is deliberately slow (cost factor

determines iterations, default ~100ms). It also automatically generates a random salt per password, so two users with the same password get different hashes. `checkpw()` uses constant-time comparison to prevent timing attacks.

Q: What would you improve if you had more time?

Three things: (1) Replace the word-based chunker with a semantic sentence-boundary chunker using spaCy for better context coherence. (2) Add re-ranking — retrieve top-20 chunks, then use a cross-encoder model to re-rank and keep top-4, improving precision. (3) Switch from Streamlit to FastAPI + React for production — Streamlit's single-threaded model doesn't support concurrent users.

Chapter 2: FitAI Pro

Flask + Groq + LLaMA 3.3 70B + SQLite + Plotly + Gunicorn

2.1 Project Overview & Elevator Pitch

FitAI Pro is a full-stack AI-powered fitness coaching and health tracking web application. Users set up a health profile — age, weight, goal weight, activity level, fitness goal, gym access, and diet preference — and the AI uses that profile to generate fully personalized workout plans, diet plans, and 6-week training programs. Beyond AI generation, the app also functions as a daily health tracker: users log weight, calories, water intake, sleep, steps, and workouts, and the analytics module converts that data into interactive Plotly charts and a Fitness Score out of 100. All tracked data is exportable as CSV or JSON.

The backend is Flask structured with the Application Factory pattern and 7 Blueprints — auth, dashboard, profile, tracking, ai_coach, analytics, and export — each owning its own routes. All AI features run through a single AEngine class backed by Groq's LLaMA 3.3 70B, with temperature tuned per feature: 0.7 for plans, 0.6 for analysis, 0.9 for motivation. The database is an 11-table SQLite schema in 3rd Normal Form — separate log tables per metric, each with a user_id foreign key and a dedicated index for O(log n) query performance. PRAGMA foreign_keys=ON enforces referential integrity at the connection level. Auth uses Flask-bcrypt and the app is deployed via Gunicorn with an app.yaml for Google App Engine.

One-Line Pitch

■ "FitAI Pro is a full-stack fitness SaaS — 7 Flask Blueprints, 11-table normalized SQLite schema with indexed foreign keys, Groq LLaMA 3.3 70B powering 7 AI features, Plotly analytics dashboard with a custom Fitness Score algorithm, bcrypt auth, and Gunicorn production deployment. Every component is built from scratch with deliberate architectural decisions I can fully defend."

Problem Statement

Generic fitness applications provide the same plans to all users regardless of their individual profile. FitAI Pro addresses this by injecting the user's complete health profile into every AI prompt — ensuring plans are tailored to the individual's age, current weight, goal, equipment availability, and dietary preference. Progress analysis and weekly reports are generated from the user's own logged data, making feedback data-driven and specific rather than generic.

Feature Modules (7 Blueprints)

Blueprint	URL Prefix	Key Features
-----------	------------	--------------

auth	/auth	Register, Login, Logout — bcrypt + Flask session
dashboard	/dashboard	Stats overview, daily summary, motivation widget
profile	/profile	Health profile, BMI calculation, goal setting
tracking	/tracking	Log weight, calories, water, sleep, steps, workouts
ai_coach	/ai	Chat, workout plans, diet plans, 6-week programs
analytics	/analytics	Plotly charts, fitness score, habit analysis
export	/export	Download data as CSV, JSON, or formatted reports

Full Feature List — What FitAI Pro Actually Does

- User registration and login with bcrypt password hashing and Flask session management
- Health profile setup: age, gender, height, weight, goal weight, activity level, fitness goal, diet preference, medical notes
- Real-time BMI calculation and categorization displayed on the profile page
- Daily workout logging: type, duration (minutes), calories burned, exercises list, personal notes
- Daily weight logging with timestamp — feeds weight trend chart and progress calculation
- Daily calorie logging: consumed vs burned with meal details text field
- Daily water intake logging in milliliters with running daily total
- Daily sleep logging: hours slept and quality rating (excellent/good/fair/poor)
- Step count logging: daily steps with logged_at timestamp
- AI Workout Plan Generator: type, duration, equipment → structured markdown plan with warm-up, exercises (sets×reps, rest, form cues), cool-down
- AI Diet Plan Generator: diet type, target calories → full-day meal plan with macro breakdown per meal
- AI 6-Week Program: muscle gain / weight loss / home / gym → periodized Foundation→Building→Peak phases
- AI Progress Analyzer: uses real 30-day tracked data (weight, workouts, sleep, water, calories) for honest analysis
- AI Weekly Report: achievements, areas for improvement, next-week goals from actual weekly data

- AI Motivational Messages: high-temperature (0.9) short energetic prompts, different every time
- AI Chat Assistant: multi-turn conversational fitness Q&A; with user profile context injected
- Fitness Score (0-100): weighted formula — 40% consistency + 30% weight progress + 30% habits
- Interactive Plotly Charts: weight trend (line), calorie balance (bar), sleep quality (scatter), workout frequency (histogram)
- Habit Analysis: calculates average sleep hours, average water intake, workout frequency per week
- Achievement Tracking: detects milestones like workout streaks, weight loss milestones, hydration goals
- Data Export: download all tracked data as CSV, JSON, or formatted AI-generated summary report
- Custom error pages: 404 and 500 with branded dark glassmorphism styling
- Unicorn WSGI deployment with app.yaml for Google App Engine cloud deployment

Project Statistics at a Glance

Flask Blueprints	DB Tables	DB Indexes	AI Features	Tracked Metrics
7	11	10	7	6 (weight/calories/water/sleep/steps/workouts)

2.2 Tech Stack — Every Choice Justified

Technology	Role	Why This Choice
Flask 3.1.1	Web Framework	Application factory pattern (create_app()), Blueprint-based modular routing, Jinja2 templating. Lightweight vs Django — no ORM overhead, full control over SQL.
Groq API (groq 0.25.0)	AI Inference	Same ultra-fast inference as RAG project. AEngine class centralizes all 7 AI features. Exponential backoff retry (3 attempts, 2^n sleep) handles rate limits gracefully.
LLaMA 3.3 70B versatile	Language Model	Different temperature per use case: 0.7 for workout/diet plans (creative variation), 0.6 for analysis (structured), 0.9 for motivation (energetic). max_tokens varies: 256 for motivation, 4096 for 6-week programs.
SQLite + sqlite3	Database	11 tables, 10 indexed foreign keys, PRAGMA foreign_keys=ON, Row factory for dict-like access, per-request connection via Flask g object and teardown_appcontext.
Flask-bcrypt 1.0.1	Auth Security	Password hashing with adaptive cost. Flask extension wrapper integrates cleanly with app factory. Session stores user_id, not username.
Plotly 6.1.2	Data Visualization	Interactive charts: weight trend (line), calorie balance (bar), sleep quality (scatter), workout frequency (histogram). JSON-serialized to Jinja2 templates.
Pandas 2.2.3	Data Processing	Analytics service aggregations, CSV export, date-range filtering, rolling averages for trend analysis.

Gunicorn 23.0.0	WSGI Server	Production deployment. app.yaml for Google App Engine. Multi-worker process model vs Flask dev server. app = create_app() at module level for Gunicorn discovery.
python-dotenv	Config Management	Config class pattern: os.environ.get with fallbacks. SECRET_KEY falls back to os.urandom(32).hex(). DATABASE_PATH is absolute path relative to basedir.
Jinja2 Templates	Frontend	Base template with navigation, per-blueprint child templates. Context processors inject user data. Static files: Inter font, custom CSS glassmorphism, vanilla JS for chart rendering.

2.3 System Architecture — Application Factory Pattern

FitAI uses Flask’s Application Factory pattern via create_app(). This is a best practice that enables testing (create multiple app instances with different configs), avoids circular imports (blueprints import from extensions, not from app), and supports deployment tools like Gunicorn that need a callable returning an app.

Component	Type	Responsibility
app.py → create_app()	Factory	Config loading, DB init, Blueprint registration, Error handlers
config.py → Config class	Configuration	SECRET_KEY, GROQ_API_KEY, DATABASE_PATH, SESSION_LIFETIME
database/db.py	DB Layer	get_db() per-request connection, g-object lifecycle, PRAGMA FK ON
database/schema.sql	Schema	11 tables, IF NOT EXISTS, 10 indexes, ON DELETE CASCADE
auth/routes.py	Auth Blueprint	Register/Login/Logout routes, session management

routes/*.py	Feature Blueprints	dashboard, profile, tracking, ai_coach, analytics, export
services/ai_engine.py	AI Layer	AIEngine class, _call_api() with retry, 7 AI methods
services/analytics_service.py	Analytics	Fitness score (0-100), habit analysis, achievement tracking
utils/decorators.py	Middleware	@login_required decorator for route protection
templates/base.html	UI Shell	Navigation, session context, Plotly JS CDN

2.4 Database Design — 11-Table Normalized Schema

The schema follows 3rd Normal Form (3NF) with proper foreign key relationships and ON DELETE CASCADE for referential integrity. Every child table has a user_id foreign key with a dedicated index for O(log n) lookup performance.

Table	Purpose & Key Columns
users	Core auth — username (UNIQUE), email (UNIQUE), password_hash, timestamps
profiles	1:1 with users — age, gender, height, weight, goal_weight, fitness_goal, diet_preference
fitness_goals	N:1 with users — goal_type, target_value, current_value, start/target dates, status
workout_logs	N:1 with users — workout_type, duration_minutes, calories_burned, exercises (JSON TEXT)
weight_logs	N:1 with users — weight (REAL), logged_at timestamp
calorie_logs	N:1 with users — calories_consumed, calories_burned, meal_details (TEXT)
water_logs	N:1 with users — amount_ml (REAL), logged_at
sleep_logs	N:1 with users — sleep_hours (REAL), sleep_quality (TEXT enum)
step_logs	N:1 with users — steps (INTEGER), logged_at
ai_reports	N:1 with users — report_type, content (full AI markdown), created_at
chat_history	N:1 with users — role (user/assistant), message, created_at

Indexing Strategy

Every log table has CREATE INDEX IF NOT EXISTS idx__user_id ON (user_id). This converts user-scoped queries from O(n) full table scan to O(log n) B-tree lookup. Critical for analytics queries that JOIN or scan multiple tables per user. PRAGMA foreign_keys=ON is set per-connection in get_db() to enforce referential integrity at the SQLite level — orphan rows are impossible.

2.5 AI Engine — Feature Breakdown

The AIEngine class is the single gateway for all AI functionality. It follows the Single Responsibility Principle — each public method handles one feature. The `_call_api()` private method centralizes error handling, retry logic, and model configuration.

Method	Temp	MaxTok	Output Structure
<code>generate_workout_plan()</code>	0.7	2048	Structured markdown: Warm-Up, Main Workout (sets×reps, rest, form cues), Cool-Down, Notes
<code>generate_diet_plan()</code>	0.7	2048	Full day: macro targets, 5 meals with portions and P/C/F breakdown, hydration
<code>generate_motivation()</code>	0.9	256	2-3 sentence energetic message, high temperature for variety
<code>analyze_progress()</code>	0.6	2048	30-day data analysis: weight history, workout count, sleep, water, calories
<code>generate_weekly_report()</code>	0.6	2048	Weekly overview, achievements, improvements, recommendations, next-week goals
<code>chat_response()</code>	0.7	1024	Conversational with profile context + chat history for multi-turn memory
<code>get_program()</code>	0.7	4096	6-week periodized program: Foundation → Building → Peak phases + nutrition

Exponential Backoff Retry

The `_call_api()` method wraps Groq calls in a 3-attempt retry loop. On rate limit errors (HTTP 429), it sleeps for 2^{attempt} seconds (1s, 2s, 4s) before retrying — exponential backoff prevents thundering herd. Non-rate-limit

errors are logged and return a graceful fallback message. This makes the AI features resilient without crashing the user's session.

Fitness Score Algorithm — AnalyticsService

The `calculate_fitness_score()` method produces a 0-100 score from three weighted components:

Component	What It Measures	Formula
Consistency (40%)	Days with at least 1 log entry in past 30 days	<code>min(tracking_days/30 * 100, 100)</code>
Progress (30%)	Weight movement toward goal weight	<code>achieved/total_needed * 100</code> (direction-aware)
Habits (30%)	Sleep quality, water intake, workout frequency	<code>analyze_habits()</code> sub-score

2.6 Interview Q&A; — FitAI Pro

Q: Why Flask over Django for this project?

Django's ORM adds abstraction that hides SQL performance issues. For a data-heavy analytics app I wanted full SQL control — I write raw queries optimized with indexes. Flask's Blueprint system gives the same modularity as Django apps without the magic. The Application Factory pattern with `create_app()` gives testability and Gunicorn compatibility. Django would be better for teams; Flask is better for demonstrating SQL and architecture knowledge.

Q: Explain your database normalization approach.

The schema is in 3NF. Users table holds only auth data. Profiles is a separate 1:1 table (one user, one profile) — keeps the users table lean for fast authentication queries. All activity logs are separate tables (workout_logs, weight_logs, etc.) rather than one fat 'activities' table — this makes queries simpler and indexes more effective. ON DELETE CASCADE means deleting a user automatically cascades through all 9 child tables.

Q: How does your Application Factory pattern work?

`create_app()` is called at module level in `app.py`, returning a configured Flask instance. Blueprints are registered inside the factory — this prevents circular imports because blueprints don't import 'app' directly, they use 'current_app' from Flask's application context. Gunicorn discovers the app via 'app = `create_app()`' at module level. Testing can call `create_app()` with a test config to get an isolated test instance.

Q: How do you handle concurrent database access?

SQLite is used via Flask's `g` object — `get_db()` creates one connection per request, stored in `g`, and closed by `close_db()` registered as a `teardown_appcontext` handler. This means each HTTP request gets its own connection, avoiding connection sharing across threads. SQLite's WAL (Write-Ahead Log) mode handles concurrent reads well. For production scale, the same code structure would work with PostgreSQL by swapping `sqlite3` for `psycopg2`.

Q: How does the AI Engine handle different use cases with the same LLM?

The key is the system prompt and temperature. For workout plans: system prompt = 'certified personal trainer', temperature=0.7 (some creativity in exercise selection). For analysis: temperature=0.6 (more deterministic, data-driven). For motivation: temperature=0.9 (maximum variety). The `_profile_summary()` helper builds a consistent user context block injected into every system prompt, personalizing all 7 features to the user's age, weight, goals, and preferences.

Q: What security measures did you implement?

Four layers: (1) bcrypt password hashing with Flask-bcrypt — adaptive cost factor, auto-salted. (2) @login_required decorator on all protected routes — redirects to login if session missing. (3) All database queries use parameterized statements (?, ?) — impossible to SQL inject. (4) SECRET_KEY for session signing — Flask sessions are HMAC-signed cookies, tamper-proof. In production I'd add CSRF tokens and rate limiting on auth endpoints.

Q: How would you scale FitAI to 10,000 users?

Three changes: (1) Replace SQLite with PostgreSQL — connection pooling with pgbouncer, JSONB for exercises column, TimescaleDB extension for time-series log queries. (2) Replace Gunicorn single-instance with a load-balanced cluster behind Nginx, multiple Gunicorn workers. (3) Cache AI responses — LRU cache for identical profile + request combinations, since workout plans for similar profiles are largely the same. Redis for session storage instead of server-side sessions.

Chapter 3: Cross-Project Comparison & Common Topics

3.1 Side-by-Side Comparison

AI RAG Chatbot	FitAI Pro
Project: AI RAG Chatbot	Project: FitAI Pro
Framework: Streamlit	Framework: Flask
Auth: Custom bcrypt + SQLite	Auth: Flask-bcrypt + Blueprint sessions
AI: RAG pipeline (retrieve then generate)	AI: Direct generation with profile context
Storage: ChromaDB (vectors) + SQLite (data)	Storage: SQLite only (11 tables, indexed)
Embedding: sentence-transformers (local)	Embedding: N/A (no document retrieval)
UI Style: Dark glassmorphism, chat-first	UI Style: Dark glassmorphism, multi-page SaaS
Deployment: Streamlit Cloud / local	Deployment: Gunicorn / Google App Engine
Architecture: Modular pipeline (6 stages)	Architecture: Application Factory + Blueprints
Key Skill: Vector search, embeddings, RAG	Key Skill: Database design, full-stack, analytics

3.2 Common Interview Topics — Generative AI

Q: What is the difference between RAG and fine-tuning?

Fine-tuning retrains model weights on domain data — expensive (GPU compute), static (data snapshot), and can cause catastrophic forgetting. RAG retrieves fresh context at inference time — cheap, always current, and doesn't modify the model. RAG is preferred when documents change frequently or when grounding/attribution matters. Fine-tuning is better for style/format adaptation.

Q: What is temperature in LLMs and how did you use it?

Temperature scales the logit distribution before softmax sampling. Temperature=0 → always picks highest probability token (deterministic). Temperature=1 → original distribution. Temperature>1 → more random. I used 0.2 for RAG answers (factual, grounded), 0.7 for workout/diet plans (creative variation), 0.9 for motivational messages (high energy, diverse output).

Q: What is HNSW and why does ChromaDB use it?

Hierarchical Navigable Small World is a graph-based approximate nearest neighbor algorithm. It builds a multi-layer graph where top layers have long-range connections (highway-like) and bottom layers have fine-grained connections. Search starts at the top layer and greedily navigates toward the query vector. Complexity: $O(\log n)$ vs $O(n)$ for brute-force. ChromaDB uses cosine similarity in the HNSW metric space.

Q: Explain the concept of embedding space.

An embedding model maps text to points in high-dimensional space (384 dims for MiniLM). Semantically similar texts are mapped to nearby points. 'King - Man + Woman $\approx$ Queen' is the classic example. In my system, when a user asks 'what is the side effect of aspirin?' and a document chunk says 'aspirin may cause stomach bleeding', their embedding vectors will be geometrically close even though they share no words.

Chapter 4: Behavioral & HR Questions

4.1 STAR Method Answers

Q: Tell me about a technical challenge you faced and how you solved it.

Situation: In the RAG chatbot, after long conversations the Groq API was returning token limit errors, crashing the app mid-conversation.

Task: Fix it without losing conversation quality or forcing users to restart.

Action: I implemented `trim_history()` — keeps the last 20 messages, with an orphan-check to ensure the trimmed history always starts with a user message. Added specific error handling to catch 'rate_limit' and 'token' error strings and show user-friendly recovery messages.

Result: App handles arbitrarily long conversations gracefully. Users see a clear 'Click Clear Chat to start fresh' message instead of a crash.

Q: Describe a project where you had to make important architectural decisions.

Situation: When building FitAI Pro, I needed to choose between SQLite and PostgreSQL, and between Django and Flask.

Task: Choose the right stack for a demo-scale fitness SaaS that I could deploy quickly and explain deeply in interviews.

Action: Chose Flask (full SQL control, Blueprint modularity, Application Factory) and SQLite (zero config, 11-table schema with indexes). Designed the schema in 3NF with ON DELETE CASCADE and per-table indexes upfront, not as an afterthought.

Result: Delivered a production-deployable app with Gunicorn. Can explain every architectural decision with specific reasoning.

Q: Why are you interested in Generative AI specifically?

I first encountered LLMs through the IBM SkillsBuild AI internship in Sept-Oct 2025, where I built my first RAG chatbot. What fascinated me was that GenAI isn't just about calling an API — it's about architecting the entire system: how you chunk and embed data affects retrieval quality, how you prompt affects output quality, how you manage context affects coherence. I built two full production systems from scratch without LangChain abstraction to develop deep understanding. I want to work at a company where I can apply and grow these skills.

Chapter 5: Quick Revision Cheat Sheet

Key Numbers, Terms, and One-Liners to Memorize

5.1 RAG Chatbot — Key Facts

- LLM: llama-3.3-70b-versatile via Groq API | Context: 128k tokens
- Embedding: all-MiniLM-L6-v2 | Dimensions: 384 | Library: sentence-transformers
- Vector DB: ChromaDB 0.5.5 | Index: HNSW cosine similarity
- Chunk size: 500 words | Overlap: 50 words | Top-K retrieval: 4 chunks
- Auth: bcrypt 4.2.0 | DB: SQLite | UI: Streamlit 1.36
- Temperature: 0.2 (RAG mode) | Max history: 20 messages | Max tokens: 1024
- Supported formats: PDF, DOCX, TXT, CSV, MD
- RAG stands for: Retrieval-Augmented Generation
- No LangChain — 100% custom pipeline in 6 stages
- Token limit fix: trim_history() keeps last 10 exchanges

5.2 FitAI Pro — Key Facts

- Framework: Flask 3.1.1 | Pattern: Application Factory + 7 Blueprints
- AI: Groq LLaMA 3.3 70B | Client: groq 0.25.0 | Retry: 3x exponential backoff
- Database: SQLite | Tables: 11 | Indexes: 10 | Constraints: ON DELETE CASCADE
- Auth: Flask-bcrypt 1.0.1 | Session: Flask session with SECRET_KEY signing
- Visualization: Plotly 6.1.2 | Data: Pandas 2.2.3 | Server: Gunicorn 23.0.0
- AI features: workout plans, diet plans, 6-week programs, progress analysis, weekly reports, motivation, chat
- Fitness score: 40% consistency + 30% progress + 30% habits (0-100 scale)
- Temperature per feature: 0.6 analysis, 0.7 plans/chat, 0.9 motivation
- DB per-request pattern: Flask g object + teardown_appcontext
- Deployment: Gunicorn WSGI server | Config: app.yaml for Google App Engine

5.3 Key Terms to Define Confidently

RAG	Retrieval-Augmented Generation — fetch relevant context at inference time to ground LLM answers
Embedding	Dense vector representation of text; semantically similar text = geometrically close vectors
HNSW	Hierarchical Navigable Small World — $O(\log n)$ approximate nearest neighbor graph algorithm
Cosine Similarity	Angle-based similarity between vectors; 1=identical direction, 0=orthogonal
Temperature	Softmax scaling factor controlling LLM output randomness; low=deterministic, high=creative
bcrypt	Adaptive password hashing with salt; deliberately slow to resist brute force
Application Factory	Flask pattern where <code>create_app()</code> returns configured app; enables testing + Gunicorn
Blueprint	Flask modular routing component; equivalent to Django app
3NF	Third Normal Form — every non-key attribute depends only on the primary key
PRAGMA foreign_keys=ON	SQLite directive to enforce referential integrity at connection level
Singleton Pattern	Design pattern ensuring one class instance; used for embedding model to avoid reloading
Parameterized Query	SQL with <code>?</code> placeholders; prevents SQL injection by separating code from data

5.4 Last-Minute Confidence Boosters

Before every interview, remind yourself of what you actually built:

- You built a complete RAG pipeline from scratch — no LangChain, no tutorials, real production code
- You understand every module: why 500-word chunks, why cosine similarity, why HNSW, why bcrypt
- You built a 7-blueprint Flask SaaS with a 3NF normalized 11-table schema with proper indexing
- You made deliberate technology choices and can defend each one with specific technical reasoning
- You handled real production concerns: token limits, rate limiting, error handling, concurrent DB access
- You deployed both projects: Streamlit Cloud for RAG, Gunicorn/GAE for FitAI
- You co-founded GS Tech Solutions and have real client-facing development experience

Best of luck, Gap. You built real things. Own it. ■
